

lecture 6 (CH 3 -- Transport Layer)

- **transport-layer services**

- | | 网络层 (Network Layer) | 传输层 (Transport Layer) |
|-------|---------------------------------------|---|
| 作用对象 | 主机到主机 (Host-to-Host) | 进程到进程 (Process-to-Process) |
| 代表协议 | IP (Internet Protocol) | TCP (Transmission Control Protocol) 、 UDP (User Datagram Protocol) |
| 运行位置 | 端系统和路由器/交换机 | 仅端系统 (主机和服务端本地) |
| 逻辑通信 | 负责主机之间的数据传递 | 负责主机内不同进程之间的数据传递, 依赖网络层 |
| 数据单位 | 包 (Packet) | 段 (Segment, TCP/UDP) |
| 可靠性 | 不可靠, 只“尽力而为”传递, 可能丢包/乱序 | TCP可靠有序, UDP不可靠无序, 依赖协议具体实现 |
| 服务约束 | 不保证时延和带宽 (delay/bandwidth guarantees) | 服务能力受网络层约束 (如IP不保证带宽, 传输层也难保证) |
| 服务补充 | 无 (IP本身不提供可靠和安全) | - 某些服务可以由传输层协议提供, 即使网络层没有提供对应服务。 - 例如, TCP能实现可靠数据传输和安全特性, 即使IP层不可靠。 - 可靠数据传输 (如TCP通过重传机制实现) - 安全特性 (如TLS在传输层实现加密) |
| 典型功能 | 路由与转发、分片与重组 | 端口管理、多路复用、流量/拥塞控制 (TCP) 、 错误检测 |
| 多协议支持 | 通常只IP协议 | TCP、UDP等多种协议供应用选择 |
| 通信场景 | 把包送到目标主机 | 把数据送到主机内特定进程 |

	网络层（Network Layer）	传输层（Transport Layer）
服务缺失	不保证时延和带宽	传输层也无法保证时延和带宽，只“尽力而为”

- **网络层（IP）**：只负责把包送到正确的主机，但不保证包的顺序和完整性，属于“尽力而为”服务。
- **传输层（TCP/UDP）**：在主机之间建立进程到进程的通信，TCP提供可靠有序的数据流，UDP提供简单快速但不可靠的报文传输。
 - UDP特性
 - 不可靠、无序传输（unreliable, unordered delivery）
 - 简单扩展（no-frills extension）
 - 进程到进程的数据传输和错误检查（process-to-process delivery and error checking）
 - TCP特性
 - 可靠、有序传输（reliable, in-order delivery）
 - 拥塞控制（congestion control）
 - 流量控制（flow control）
 - 连接建立（connection setup）
- **服务约束与补充**：传输层的可靠性、安全性等能力可以在网络层不可靠的情况下补充，但无法突破网络层的物理限制（如带宽、时延）。
- 传输层依赖并增强网络层的服务。
- **传输层的功能和流程**:
 - **发送端（Sender）**：
 - 当应用层要发送数据时，传输层会把这些数据“打包”成一个个**段（segment）**。
 - 如果是TCP协议，叫“TCP segment”；如果是UDP协议，叫“UDP datagram”。
 - 传输层打包好以后，把这些段交给**网络层**，由网络层负责把段分发到目标主机。
 - **接收端（Receiver）**：
 - 网络层把收到的数据传送给传输层。
 - 传输层会把这些段重新**组装还原成应用层消息**（例如，把字节流还原成完整的文件或网页）。
 - 组装好后交给应用层，由应用程序处理展示。

• multiplexing and demultiplexing

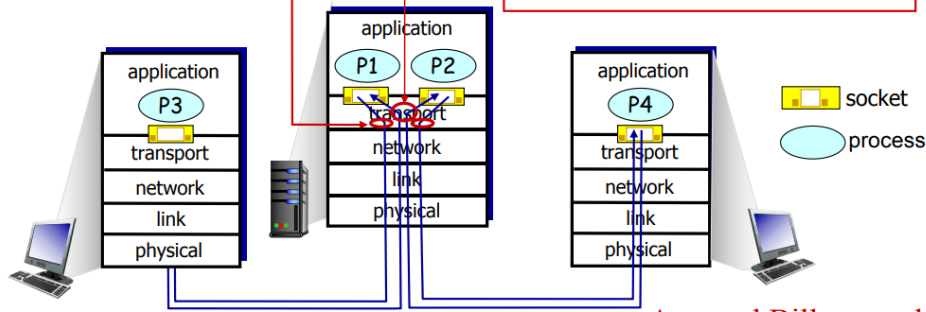
•

Multiplexing at sender:

handle data from multiple sockets, add transport header (later used for demultiplexing)

Demultiplexing at receiver:

use header info to deliver received segments to correct socket



Ann and Bill example?

- **Multiplexing (多路复用)**：在发送端，将来自多个 socket 的数据合并，通过添加传输层头部（如端口号），在同一主机上实现多个进程的数据一起发送。
- **Demultiplexing (多路分解)**：在接收端，利用传输层头部信息（如端口号），把收到的 segment 分发到正确的 socket（即正确的应用进程）。

- **技术细节**

- **socket有唯一标识符**：每个 socket 都有唯一的端口号（identifier）。
- **每个 segment 有特殊字段**：如源端口号和目的端口号，标明该数据属于哪个 socket。

- **发送主机 sending host**

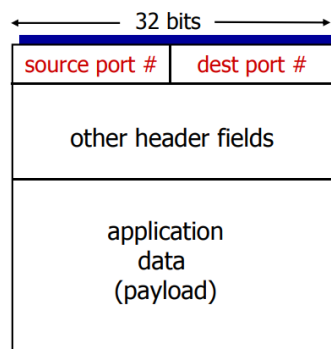
- 使用 IP 地址和端口号来确定 segment 发送到哪个 socket。

- **接收主机 receiving host**

- 网络层收到数据报后，每个数据报包含源 IP、目的 IP。
- 每个数据报都携带一个传输层 segment。

- **TCP/UDP segment 格式**

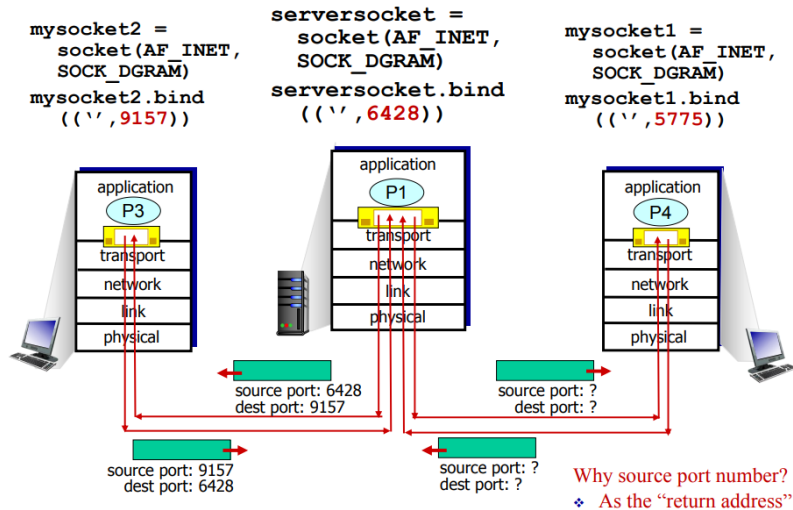
- 头部包含 source port #（源端口号）、dest port #（目的端口号）、其他头部字段，和应用数据。
- 端口号是16位，范围 0 ~ 65535，其中 0 ~ 1023 为知名端口（如HTTP 80）。
-



TCP/UDP segment format

- UDP: 无连接的多路分解 (Connectionless demux)

Connectionless demux: example



- UDP不需要建立连接，收到的数据通过端口号直接分发到对应socket。

- 基础流程

- 创建 UDP 套接字时，自动分配本地端口号；也可以手动指定（如服务器端）。
 - 示例：clientSocket = socket(AF_INET, SOCK_DGRAM)（自动分配端口）
 - 示例：clientSocket.bind(('', 19157))（手动指定端口）
- 发送数据报时，必须指定目标 IP 地址和端口号：
 - clientSocket.sendto(message.encode(), (serverName, serverPort))

- UDP 套接字唯一标识

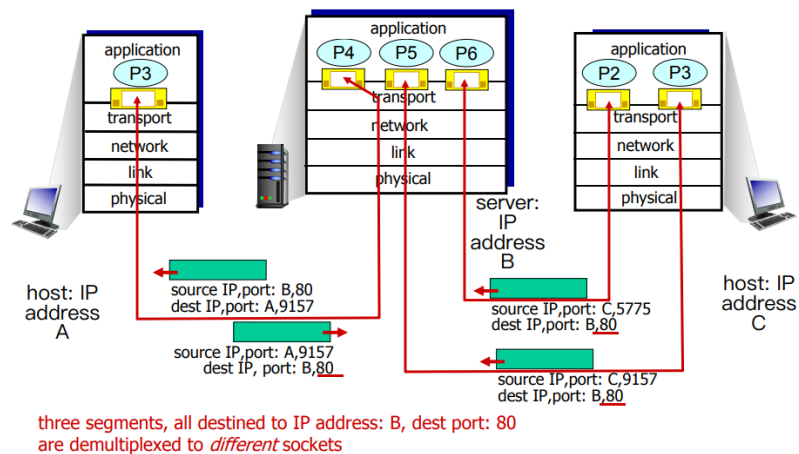
- UDP套接字由“目标IP地址 + 目标端口号”唯一确定 (two-tuple)。

- 主机收到UDP段时的流程

- 检查段中的目标端口号。
- 把UDP段交给对应端口号的socket。
- 所有目标端口号相同的数据报，无论源IP/源端口如何，都会被分发到同一个socket。

- TCP: 面向连接的多路分解 (Connection-oriented demux)

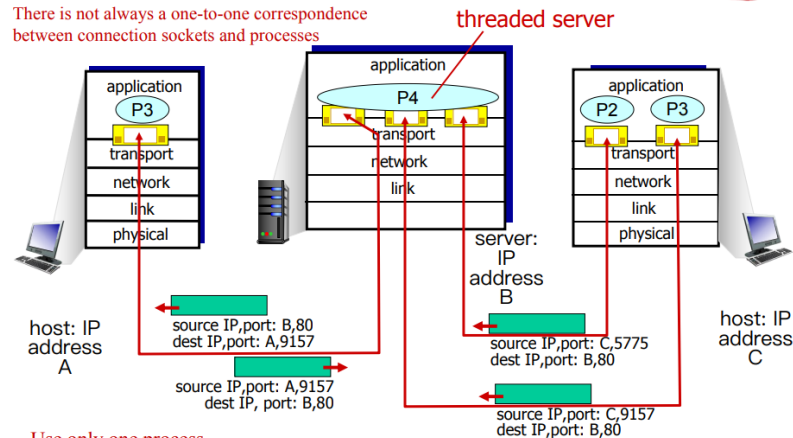
Connection-oriented demux: example



- 并不是每个连接socket都对应一个进程，现实中服务器常用**多线程 (threaded server) **模型。
- 服务器只用一个主进程（如P4），每接收到一个新的客户端连接，就创建一个新的线程和相应的连接socket。
- TCP建立连接后，每个连接由源IP、源端口、目的IP、目的端口唯一标识，数据通过专属连接分发。

Connection-oriented demux: example

There is not always a one-to-one correspondence between connection sockets and processes



Use only one process

- ❖ Create a new thread with a new connection socket for each new client connection.
- ❖ A thread can be viewed as a lightweight subprocess.

基础流程

- 服务器端创建欢迎套接字 (welcoming socket)，指定端口号，如12000。
 - `serversocket = socket(AF_INET, SOCK_STREAM)`
 - `serversocket.bind(('', 12000))`
- 客户端主动连接服务器, the request is a TCP segment with a flag bit = 1:
 - `clientsocket = socket(AF_INET, SOCK_STREAM)`

- `clientsocket.connect((ServerName, 12000))`
- 这里的 **flag bit** 指的是 **TCP头部中的控制标志位（Flags）**。
- 在TCP连接建立（三次握手）阶段，客户端第一次发给服务器的数据包叫做 **SYN段**，它的TCP头部里有一个叫 **SYN** 的flag（标志位），值被设置为1。
- **SYN=1** 表示“请求建立连接”。
- TCP头部还有其它flag，比如ACK、FIN、RST等，分别用于确认、断开等控制。
- 这句话的意思就是：当客户端要与服务器建立连接时，发送的第一个TCP数据包里，SYN标志位被置为1，服务器收到后才会进行下一步（三次握手的第一步）。
- 服务器端为每个连接创建新套接字（connection socket）：
 - `connectionsocket, addr = serversocket.accept()`
- TCP套接字标识方式
 - TCP套接字由“四元组”唯一标识：
 - 源IP地址、源端口号、目标IP地址、目标端口号（source IP, source port, dest IP, dest port）
- 多路分解原理
 - 接收端用这四个值来确认每个segment属于哪个连接和socket。
 - 服务器支持多个并发连接，每个连接对应自己的四元组和socket。
- Web服务器为每个连接的客户端都分配不同的socket（连接套接字）。
- 不管是连接建立时的TCP段，还是后面携带HTTP请求的TCP段，它们的目标端口都是80（HTTP标准端口）。
- 非持久HTTP（non-persistent HTTP）：每次浏览器请求一个网页资源（比如图片、脚本）都会重新建立一次TCP连接，服务器会为每个连接分配一个新的socket。每个请求-应答结束后，这个连接和socket就关闭。
 - 持久HTTP则是多个请求复用一個连接和socket。
- 多连接支持
 - 服务器可以同时维护多个TCP连接，每个连接有独立的connection socket。

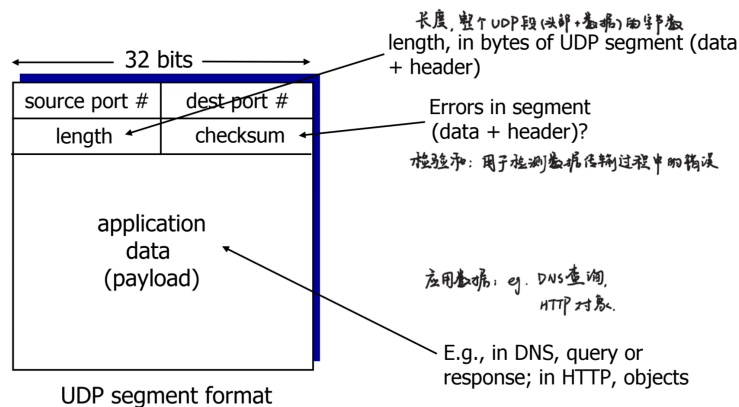
- 传输层通过 segment 的端口号把数据交给对应应用进程。

• connectionless transport: UDP

- UDP协议特点：支持多路复用/分解，轻量级错误检测。

- 如何在UDP上实现可靠传输
 - 通过应用层增加可靠性机制（如超时重传、ACK、序号等）。
 - 应用根据自身需求，设计专门的错误恢复策略。

UDP: segment header



- UDP checksum

Internet checksum: example

At the sender side, **determine the check sum** of the following three 16-bit words

```
0110011001100000
0101010101010101
1000111100001100
```

The sum of first two of these 16-bit words is

```
0110011001100000
0101010101010101
1011101110110101
```

Adding the third word to the above sum gives

```
1011101110110101
1000111100001100
1010010101100001
0100101011000010
```

Note: when adding numbers, a carryout from the most significant bit needs to be added to the result

Wrap around

The 1s complement is obtained by converting all the 0s to 1s and converting all the 1s to 0s.

Checksum: 1011010100111101

Receiver Side?

- 目标: 检测数据在传输过程中是否发生错误（如某些比特被翻转）
- 发送方流程
 - 把UDP段（包括头部和数据）看作一组16位整数
 - 将所有16位数值相加（如果有进位需回卷加到末位）
 - 取这个和的**1的补码**（即把所有0变成1，所有1变成0）
 - 把得到的校验和值填入UDP头部的checksum字段
- 接收方流程
 - 重新把整个段（收到的数据+checksum字段）当作一组16位整数加起来
 - 如果结果所有位都是1（即全是1），则认为没有检测到错误

- 如果结果中有0，则说明数据包在传输过程中发生了错误
- 为什么UDP要提供校验和（checksum）？
 - 没有保证所有链路都做了错误检测
 - 网络中的每一跳（路由器、交换机等）不一定都能做错误检测，所以不能完全依赖底层链路。
 - 比特错误可能在内存中发生
 - 在数据包传输、存储过程中，即使还没到物理传输阶段，也可能由于硬件故障发生比特翻转。
 - 功能层级的考虑
 - 某些低层功能可能冗余或效果有限，综合成本更高。把关键的错误检测放在传输层（如UDP/TCP）更有效、更可靠。